claude-windows / 7fce5410-4c57-4ec9-a054-be898bda22da

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7fce5410-4c57-4ec9-a054-be898bda22da\subagents\workflows\wf_c81dbe7a-7ff\agent-a2a4828d7b0dee66e.jsonl`
- SHA-256: `955f60ed20ef28cf48b9599ef368005f751470bddaccbb288d4c269f7d5819bb`
- Source modified: `2026-06-25T17:01:52+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_c81dbe7a-7ff`
- session_id: `7fce5410-4c57-4ec9-a054-be898bda22da`

Transcript

1. user (2026-06-25T17:01:08.867Z)

Adversarially verify this PR-review finding for PR #380 (branch o1-p11/gemini-remote-file-gc is checked out; read the real code with git diff origin/master...HEAD + Read the files). Be skeptical ? default to isReal=false / adjustedSeverity=INVALID if the finding is wrong, already handled by the code, or a non-issue. Do NOT re-run CI.

Finding [NIT] create_time handled as an RFC3339 string but the real SDK returns a datetime ? works by luck of str(); the test feeds a string so the real type is never exercised
Location: backend/providers/gemini.py:70-75 + tests/test_provider_gemini.py:_file_mock
Detail: The comment at gemini.py:70 says 'f.create_time is an RFC3339 string on the File API' and the code does datetime.fromisoformat(str(ct).replace('Z','+00:00')) (line 75). But in google-genai 2.4.0, File.create_time is typed Optional[datetime.datetime] (verified: types.File.model_fields['create_time'].annotation is Optional[datetime.datetime]) ? a real timezone-aware datetime, not a string. It happens to still parse: str(datetime_obj) yields '2026-06-25 12:00:00+00:00' (space sep, no Z) which fromisoformat accepts. So no live bug today, but the comment is factually wrong and the str()-round-trip is fragile (any non-isoformat __str__ would break it). Worse, the test's _file_mock sets create_time to a .isoformat() STRING (tests/test_provider_gemini.py), so the unit tests exercise the string path, NOT the real datetime path ? false confidence. FIX: handle the datetime directly (if isinstance(ct, datetime): created = ct; else parse), drop the misleading comment, and make at least one test feed a real datetime to match the SDK.
Suggested fix: Treat create_time as a datetime first (isinstance check), fall back to ISO-string parse; correct the 'RFC3339 string' comment; add a test case that passes a real datetime so the production type is covered.

Return isReal + adjustedSeverity (INVALID if not real) + one-paragraph reasoning grounded in the actual code.

2. assistant (2026-06-25T17:01:11.868Z)

I'll verify this finding by examining the actual code in PR #380.

3. user (2026-06-25T17:01:13.130Z)

o1-p11/gemini-remote-file-gc
f00b71e feat(O1/P11): Gemini remote-file GC ? startup orphan sweep
86705c1 chore: ruff format gemini_refine.py + trajectory_hybrid.py (#379)
b72ac85 fix(B2/P17): surface Gemini provider failures in all callers (#378)

4. user (2026-06-25T17:01:14.138Z)

diff --git a/backend/providers/gemini.py b/backend/providers/gemini.py
index 2ea0834..428bca8 100644
--- a/backend/providers/gemini.py
+++ b/backend/providers/gemini.py

```
@@ -2,6 +2,7 @@ import json
 import logging
 import os
 import time
+from datetime import datetime, timedelta, timezone
 from typing import Tuple

 from google import genai
@@ -40,6 +41,48 @@ class GeminiProvider(AIVideoProvider):
         except Exception:
             pass

+    def cleanup_orphaned_files(self, grace_sec: int = 3600) -> int:
+        """Sweep and delete video files orphaned on the Gemini File API by a previous
+        crashed run (a hard process death between ``files.upload`` and ``_safe_delete``
+        leaves the uploaded clip behind forever ? the per-call cleanup only covers paths
+        that reach a ``finally``/except).
+
+        Best-effort: lists every file, deletes any whose ``mime_type`` is a video AND whose
+        creation is older than ``grace_sec`` (so we never race a concurrent worker mid-upload).
+        Never raises; returns the count deleted. Intended to run once at startup.
+
+        Args:
+            grace_sec: only delete files older than this (default 1h). Protects a file a
+                live worker may still be processing.
+        """
+        cutoff = datetime.now(timezone.utc) - timedelta(seconds=grace_sec)
+        deleted = 0
+        try:
+            files = list(self.client.files.list())
+        except Exception as e:
+            logger.warning("[Gemini GC] failed to list remote files (skipping sweep): %s", e)
+            return 0
+        for f in files:
+            try:
+                mime = getattr(f, "mime_type", "") or ""
+                if not mime.startswith("video/"):
+                    continue
+                # f.create_time is an RFC3339 string on the File API.
+                ct = getattr(f, "create_time", None)
+                if ct is None:
+                    # No timestamp ? can't prove it's stale; leave it (safe default).
+                    continue
+                created = datetime.fromisoformat(str(ct).replace("Z", "+00:00"))
+                if created > cutoff:
+                    continue  # too recent ? a live worker may own it
+                self.client.files.delete(name=f.name)
+                deleted += 1
+            except Exception as e:
+                logger.debug("[Gemini GC] skipping a file during sweep: %s", e)
+        if deleted:
+            logger.info("[Gemini GC] deleted %d orphaned video file(s) older than %ds.",
deleted, grace_sec)
+        return deleted
+
     def analyze_video(self, video_path: str, prompt: str,
                       chunk_duration: float = 0.0,
                       label: str = "") -> Tuple[list, dict]:
```

5. assistant (2026-06-25T17:01:16.534Z)

Now let me read the actual file and the test mock to verify the claim about how `create_time` is handled and tested.

6. user (2026-06-25T17:01:17.944Z)

```
44          def cleanup_orphaned_files(self, grace_sec: int = 3600) -> int:
45              """Sweep and delete video files orphaned on the Gemini File API by a previous
46              crashed run (a hard process death between ``files.upload`` and ``_safe_delete``
47              leaves the uploaded clip behind forever ? the per-call cleanup only covers paths
48              that reach a ``finally``/except).
49
50              Best-effort: lists every file, deletes any whose ``mime_type`` is a video AND
whose
51              creation is older than ``grace_sec`` (so we never race a concurrent worker mid-
upload).
52              Never raises; returns the count deleted. Intended to run once at startup.
53
54              Args:
55                  grace_sec: only delete files older than this (default 1h). Protects a file a
56                      live worker may still be processing.
57              """
58              cutoff = datetime.now(timezone.utc) - timedelta(seconds=grace_sec)
59              deleted = 0
60              try:
61                  files = list(self.client.files.list())
62              except Exception as e:
63                  logger.warning("[Gemini GC] failed to list remote files (skipping sweep):
%s", e)
64                  return 0
65              for f in files:
66                  try:
67                      mime = getattr(f, "mime_type", "") or ""
68                      if not mime.startswith("video/"):
69                          continue
70                      # f.create_time is an RFC3339 string on the File API.
71                      ct = getattr(f, "create_time", None)
72                      if ct is None:
73                          # No timestamp ? can't prove it's stale; leave it (safe default).
74                          continue
75                      created = datetime.fromisoformat(str(ct).replace("Z", "+00:00"))
76                      if created > cutoff:
77                          continue  # too recent ? a live worker may own it
78                      self.client.files.delete(name=f.name)
79                      deleted += 1
80                  except Exception as e:
81                      logger.debug("[Gemini GC] skipping a file during sweep: %s", e)
82              if deleted:
83                  logger.info("[Gemini GC] deleted %d orphaned video file(s) older than %ds.",
deleted, grace_sec)
84              return deleted
85
86          def analyze_video(self, video_path: str, prompt: str,
87                            chunk_duration: float = 0.0,
88                            label: str = "") -> Tuple[list, dict]:
```

7. user (2026-06-25T17:01:19.273Z)

```
diff --git a/tests/test_provider_gemini.py b/tests/test_provider_gemini.py
index 141af81..5df2ef4 100644
--- a/tests/test_provider_gemini.py
+++ b/tests/test_provider_gemini.py
@@ -101,3 +101,107 @@ def test_gemini_provider_upload_fail():
         provider = GeminiProvider(api_key="dummy_api_key", model_name="gemini-2.5-flash")
         segments, metrics = provider.analyze_video("dummy.mp4", "dummy prompt")
         assert len(segments) == 0
+
+
+
+# --- O1/P11: Gemini remote-file GC (startup orphan sweep) ---
+
```

```
+def _file_mock(name, mime, create_time_iso, state="ACTIVE"):
+    f = MagicMock()
+    f.name = name
+    f.mime_type = mime
+    f.create_time = create_time_iso
+    f.state = MagicMock()
+    f.state.name = state
+    return f
+
+
+def test_cleanup_orphaned_files_deletes_stale_videos_only():
+    """Deletes video files older than the grace window; leaves non-videos and fresh files."""
+    from datetime import datetime, timedelta, timezone
+    stale = (datetime.now(timezone.utc) - timedelta(hours=2)).isoformat()
+    fresh = (datetime.now(timezone.utc) - timedelta(minutes=5)).isoformat()
+    files = [
+        _file_mock("old-vid", "video/mp4", stale),       # stale video -> deleted
+        _file_mock("fresh-vid", "video/quicktime", fresh), # fresh video -> kept (grace)
+        _file_mock("old-img", "image/png", stale),        # non-video -> kept
+        _file_mock("notts", "video/mp4", None),           # no timestamp -> kept (safe)
+    ]
+    with patch("backend.providers.gemini.genai"):
+        provider = GeminiProvider(api_key="k", model_name="m")
+        provider._client = MagicMock()
+        provider._client.files.list.return_value = iter(files)
+        deleted = provider.cleanup_orphaned_files(grace_sec=3600)
+    assert deleted == 1
+    provider._client.files.delete.assert_called_once_with(name="old-vid")
+
+
+def test_cleanup_orphaned_files_list_failure_is_swallowed():
+    """A list() failure must not raise ? the sweep is best-effort and must not block
startup."""
+    with patch("backend.providers.gemini.genai"):
+        provider = GeminiProvider(api_key="k", model_name="m")
+        provider._client = MagicMock()
+        provider._client.files.list.side_effect = RuntimeError("network")
+        assert provider.cleanup_orphaned_files() == 0
+
+
+def test_cleanup_orphaned_files_per_file_delete_failure_is_swallowed():
+    """A delete() failure on one file must not abort the sweep."""
+    from datetime import datetime, timedelta, timezone
+    stale = (datetime.now(timezone.utc) - timedelta(hours=2)).isoformat()
+    files = [
+        _file_mock("a", "video/mp4", stale),
+        _file_mock("b", "video/mp4", stale),
+    ]
+    with patch("backend.providers.gemini.genai"):
+        provider = GeminiProvider(api_key="k", model_name="m")
+        provider._client = MagicMock()
+        provider._client.files.list.return_value = iter(files)
+        # First delete raises, second succeeds ? _safe_delete swallows it.
+        provider._client.files.delete.side_effect = [RuntimeError("boom"), None]
+        deleted = provider.cleanup_orphaned_files(grace_sec=3600)
+    assert deleted == 1  # only the second file counted
+
+
+
+# --- O1/P11: main.py startup wiring (_maybe_sweep_gemini_orphans) ---
+
+def test_maybe_sweep_skips_non_gemini_provider(monkeypatch):
+    """A non-Gemini provider must short-circuit (no client built, no sweep)."""
+    import backend.main as main_mod
+    cfg = MagicMock()
```

```
+    cfg.ai_provider = "openai"
+    called = []
+    monkeypatch.setattr(main_mod.provider_registry, "get",
+                        lambda *a, **k: called.append(k) or MagicMock())
+    monkeypatch.setenv("GEMINI_API_KEY", "k")
+    main_mod._maybe_sweep_gemini_orphans(cfg)
+    assert called == []  # provider never instantiated
+
+
+def test_maybe_sweep_skips_when_no_api_key(monkeypatch):
+    """No GEMINI_API_KEY -> no sweep (can't auth)."""
+    import backend.main as main_mod
+    cfg = MagicMock()
+    cfg.ai_provider = "gemini"
+    monkeypatch.delenv("GEMINI_API_KEY", raising=False)
+    called = []
+    monkeypatch.setattr(main_mod.provider_registry, "get",
+                        lambda *a, **k: called.append(k) or MagicMock())
+    main_mod._maybe_sweep_gemini_orphans(cfg)
+    assert called == []
+
+
+def test_maybe_sweep_runs_and_swallows_errors(monkeypatch):
+    """Gemini + key present -> sweep runs; a provider error is swallowed (never blocks
startup)."""
+    import backend.main as main_mod
+    cfg = MagicMock()
+    cfg.ai_provider = "gemini"
+    cfg.ai_model = "gemini-2.5-flash"
+    provider = MagicMock()
+    provider.cleanup_orphaned_files.side_effect = RuntimeError("api down")
+    monkeypatch.setattr(main_mod.provider_registry, "get", lambda *a, **k: provider)
+    monkeypatch.setenv("GEMINI_API_KEY", "k")
+    # Must not raise.
+    main_mod._maybe_sweep_gemini_orphans(cfg)
+    provider.cleanup_orphaned_files.assert_called_once()
```

8. assistant (2026-06-25T17:01:21.683Z)

The finding is confirmed against the actual code. Let me verify the SDK type claim ? whether `File.create_time` is actually typed as a datetime in the installed google-genai version.

9. user (2026-06-25T17:01:25.788Z)

```
genai version: 2.4.0
create_time annotation: typing.Optional[datetime.datetime]
```

10. assistant (2026-06-25T17:01:31.772Z)

The SDK claim is verified: google-genai 2.4.0 types `File.create_time` as `Optional[datetime.datetime]`. Now let me verify the str() round-trip claim ? that a real timezone-aware datetime parses correctly through the current code, and that the test only exercises the string path.

11. user (2026-06-25T17:01:34.956Z)

```
str(dt) = '2026-06-25 15:01:34.933081+00:00'
parsed OK = 2026-06-25 15:01:34.933081+00:00
equal to original (sans micro precision): True
str(isoformat_str) = '2026-06-25T15:01:34.933081+00:00'
parsed OK = 2026-06-25 15:01:34.933081+00:00
```

12. assistant (2026-06-25T17:01:41.556Z)

All claims are verified. The finding is technically accurate on every factual point:

1. **The comment is factually wrong**: gemini.py:70 says `create_time is an RFC3339 string`, but google-genai 2.4.0 types it as `Optional[datetime.datetime]` (verified via introspection).

2. **It works "by luck of str()"**: A real tz-aware datetime ? `str(dt)` gives `'2026-06-25 15:01:34+00:00'` (space separator, no `Z`), which `fromisoformat` accepts. Confirmed parsing works.

3. **The test only exercises the string path**: `_file_mock` sets `create_time` to `.isoformat()` strings, so the real datetime type is never exercised in tests ? false confidence is accurate.

Now I'll assess severity. The finding correctly self-labels as NIT and explicitly states "no live bug today." The code works correctly with both the real SDK type and the test's string type. This is a documentation-accuracy issue plus a test-coverage gap, not a correctness bug.

13. user (2026-06-25T17:01:47.395Z)

Structured output provided successfully

14. assistant (2026-06-25T17:01:52.243Z)

Finding verified as **real, severity NIT**. The relevant files are `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\providers\gemini.py` (lines 70-75) and `C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_provider_gemini.py` (`_file_mock`). All factual claims hold: the comment is wrong (SDK 2.4.0 types `create_time` as `Optional[datetime.datetime]`, not a string), the `str()` round-trip works correctly so there's no live bug, and the tests only feed isoformat strings so the production datetime type is never exercised. It's a documentation-accuracy + test-coverage nit with no functional impact.